

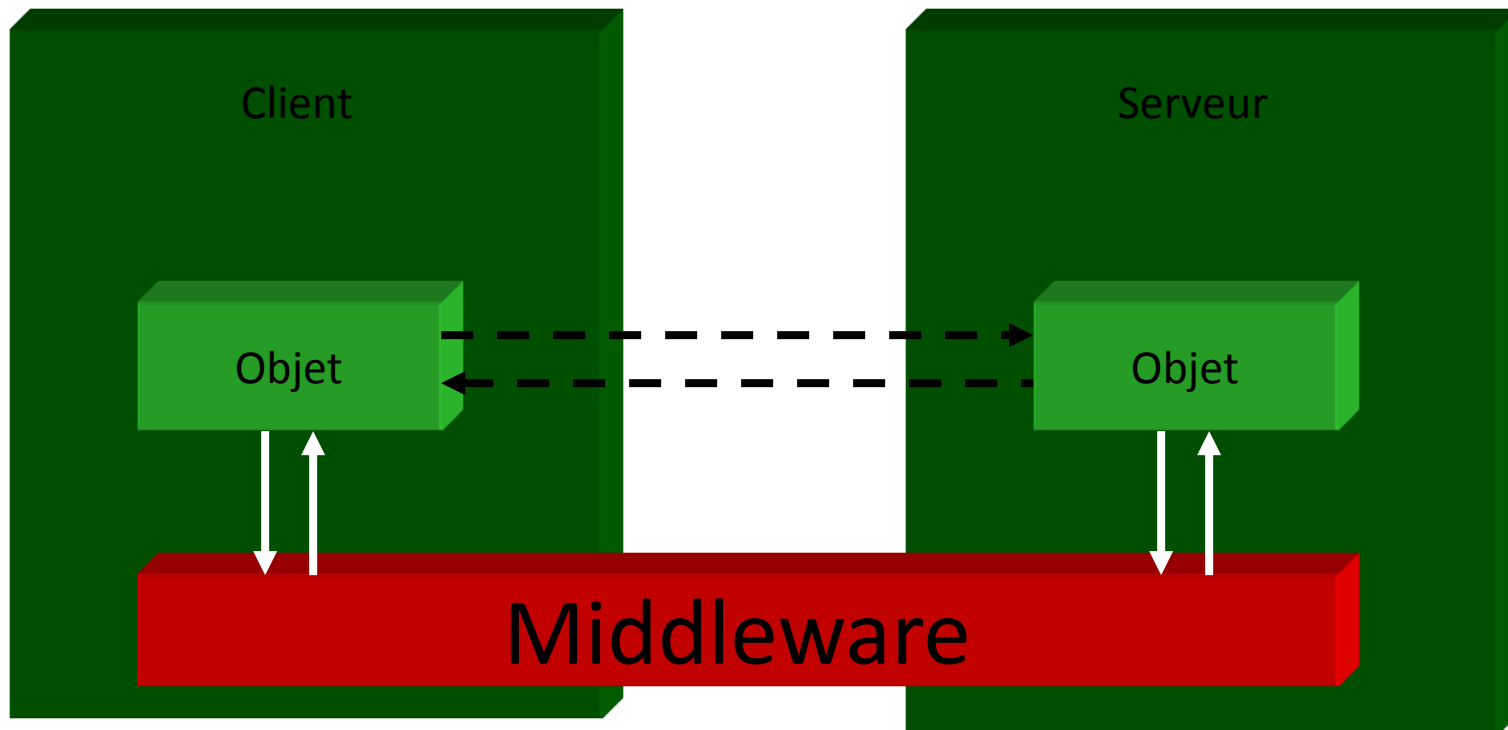
CORBA

Common Object Request Broker Architecture

Contexte

- Applications distribuées
- Objets répartis
- Autres technologies :
 - RPC
 - COM+
 - RMI
 - .NET

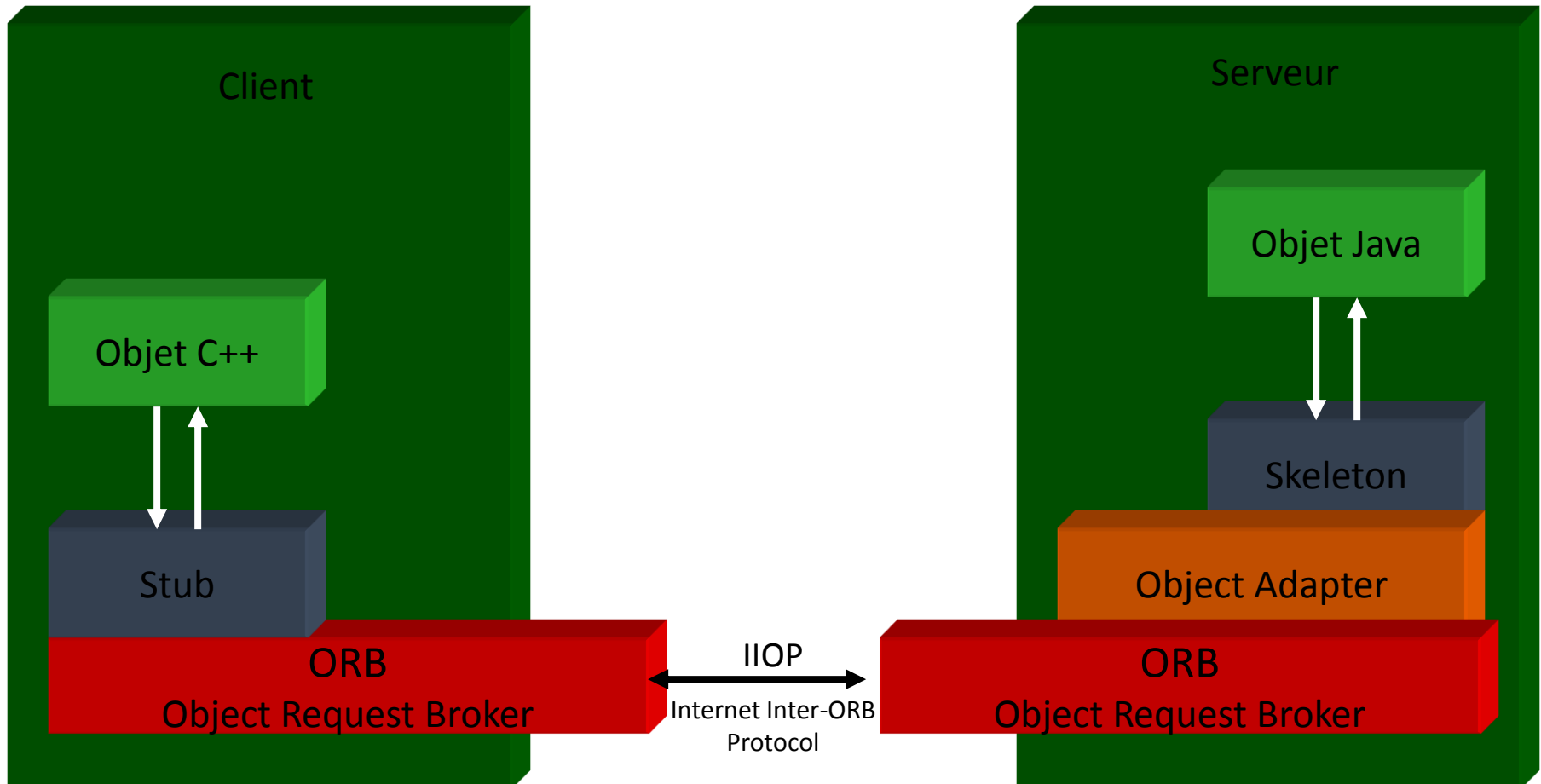
Principe simplifié



Object Management Group (OMG)

- Créée en 1989
- But non lucratif
- Plus de 850 membres (Sun, IBM, Microsoft, ...)
- Crée et maintient les spécifications
 - CORBA
 - UML

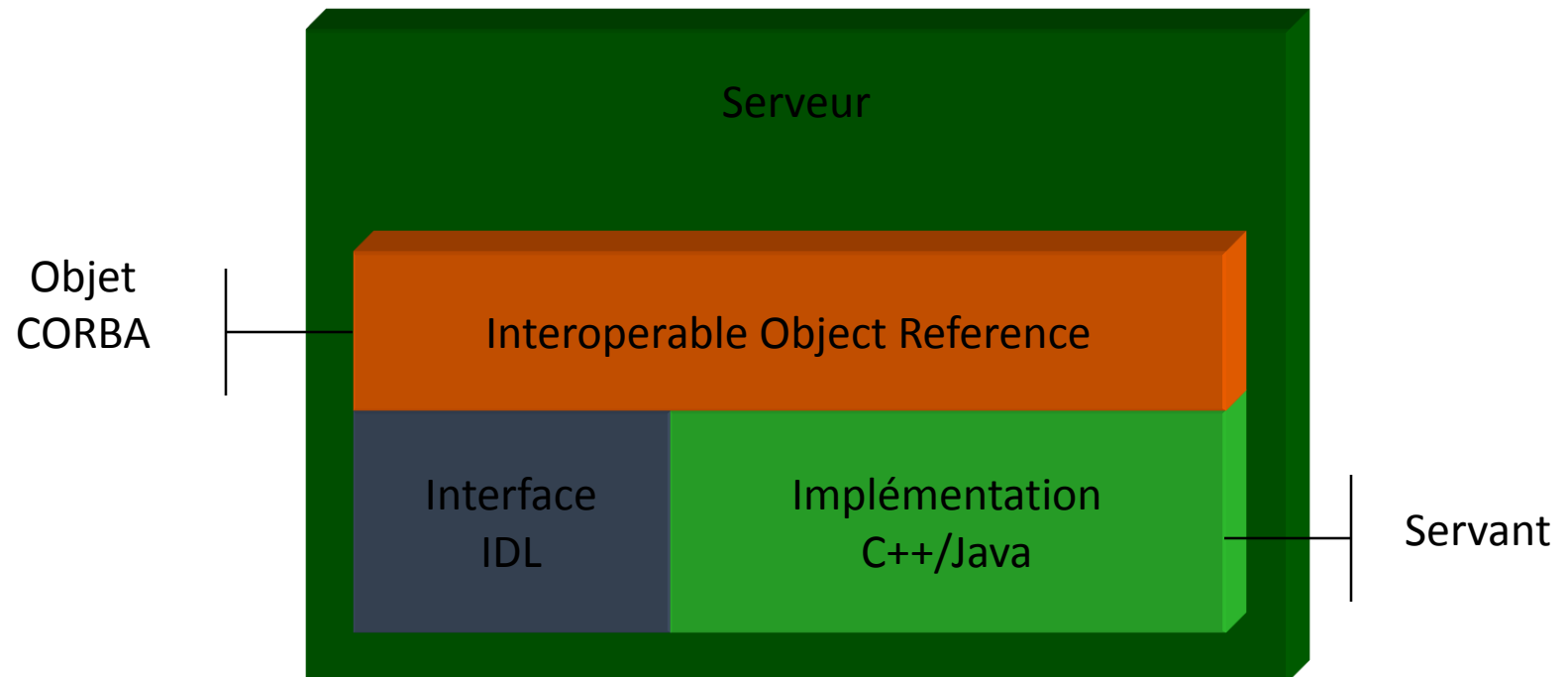
Architecture de CORBA



Composants

- Stub : partie cliente
- Skeleton : partie serveur
- Object Adapter : enregistrement des objets
- ORB : passage de message
- IIOP : Internet Inter-ORB Protocol

Objet CORBA



Interface Definition Langage (IDL)

- Description des interfaces
- Indépendant du langage d'implémentation
- Indépendant de la plate-forme client
- Indépendant de la plate-forme serveur

- Ressemble beaucoup au C++

Example

```
module example {  
    interface monExample {  
        void methode1();  
        long methode2();  
        void methode3(in long param, out long result);  
    };  
};
```

Stub (souche)

- Code client
- Interface entre objet et ORB
- Traduit les invocations sur l'objet serveur
 - > marshalling
- Traduit les messages en valeurs de retour
 - unmarshalling

Skeleton (squelette)

- Code serveur
- Interface entre implémentation et ORB
- Traduit les invocations client vers l'implémentation
 - > unmarshalling
- Traduit la valeur de retour en message vers client
 - > marshalling

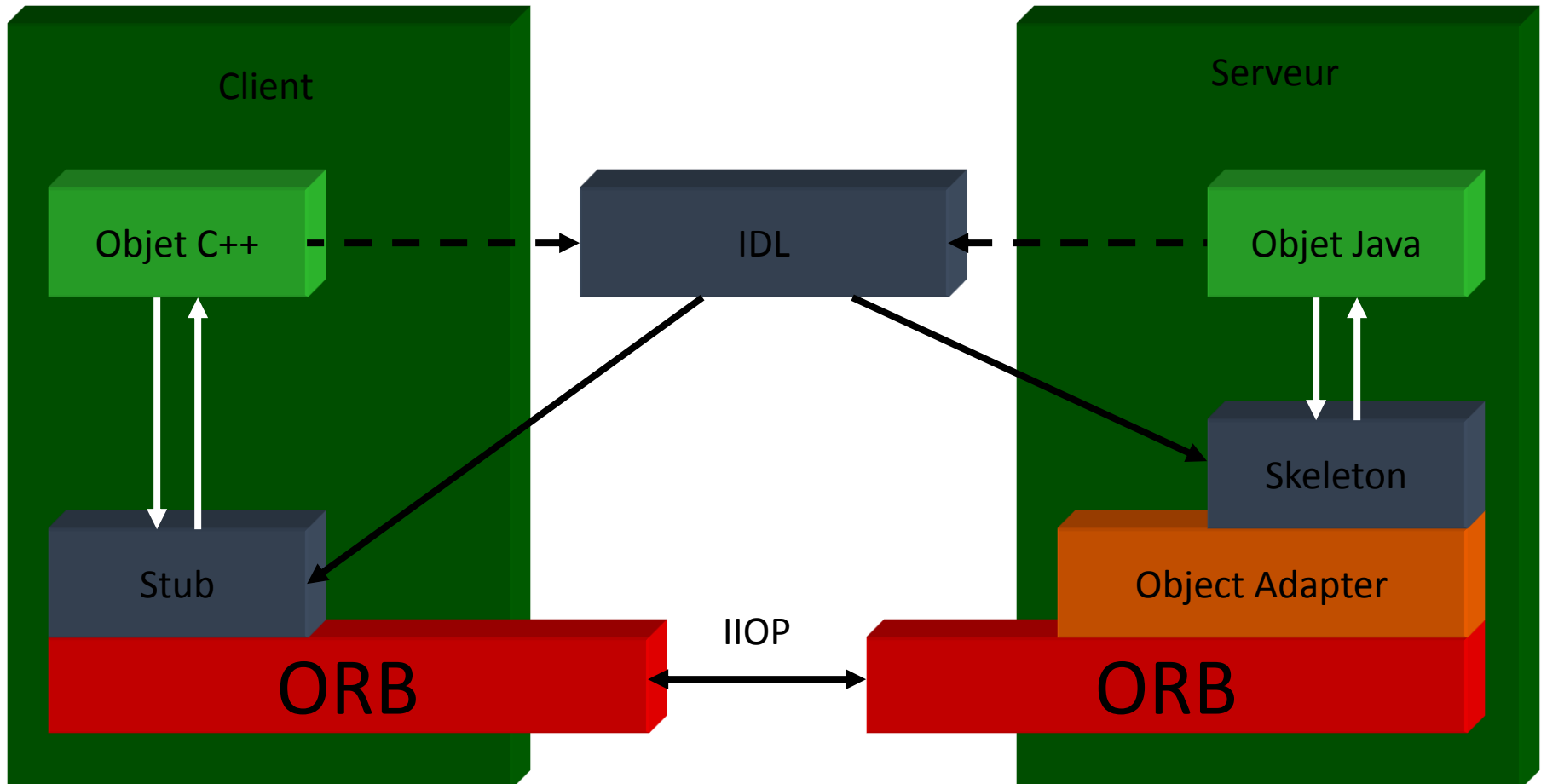
Object Request Broker (ORB)

- Transporte les messages entre les objets
- Relie les stubs aux skeletons correspondants et vice-versa
- Bus à objets
- Communications inter-ORBs :
 - GIOP (General Inter-ORB Protocol)
 - IIOP (Internet Inter-ORB Protocol) (GIOP on TCP/IP)

Object Adapter (OA)

- Enregistre et gère les implémentations
- Activation et désactivation des objets
- Invocation des méthodes
- Authentification du client / contrôle d'accès
- Différents types :
 - BOA – Basic Object Adapter
 - POA – Portable Object Adapter

Architecture de CORBA



Création d'une application CORBA

- 1 – Définir l'interface IDL
- 2 – Compiler l'interface IDL
- 3 – Créer l'implémentation de l'interface IDL
- 4 – Créer le serveur :
 - > publication de l'objet CORBA
- 5 – Créer le client :
 - > appel de l'objet CORBA

Exemple IDL

```
module example e {  
    interface monExample {  
        void methode1();  
        long methode2();  
        void methode3(in long param, out long result);  
    };  
};
```

Module

```
module monModule {
```

```
...
```

```
};
```

- espace de nom
 - C++ : namespace
 - Java : package

Interface

```
interface monInterface {  
    ...  
};
```

- Classe

- C++ : class
- Java : interface

Méthode

```
void methode(in long param, out long result);
```

- Méthodes : comme en C++ et Java, sauf
 - in : paramètre utilisé en entrée (lu, non modifié)
 - out : paramètre utilisé en sortie (non lu, modifié)
 - inout : paramètre utilisé en entrée et en sortie (lu et modifié)

Syntaxe

- Commentaire : comme C++ et Java
 - `//` : jusqu'à la fin de la ligne
 - `/* ... */` : bloc de commentaire
- Préprocesseurs : `#define`, `#include`, `#ifdef`, `#endif`
- Alias : `typedef`
- Possibilité de définir des attributs
- Constantes : `const`

Types primitifs

- void
- short
- long
- long long
- float
- double
- boolean

Types complexes

- string
- struct
- enum
- union
- any

Services de recherche d'objets

- Service de nommage (Naming)
 - Recherche d'objet par nom
 - pages blanches
- Service vendeur (Trader)
 - Recherche d'objet par propriété
 - pages jaunes

Besoin du Naming

- Référence d'un objet : IOR
 - Fichier partagé, ...
- Service à la DNS :
 - Service accessible par le bus ORB
 - Service standard entre ORBs
 - Un nom spécifique <-> un objet corba
 - Gère les contextes de nom

Exemple de code minimal CORBA en java

Icarre.idl

```
interface Icarre {  
    long carre(in long source);  
}
```

IcarreImpl.java

```
import org.omg.CORBA.*;  
  
public class IcarreImpl extends IcarreImplBase {  
    public int carre (int source) {  
        return source*source;  
    }  
}
```

Exemple de code minimal CORBA en java

Serveur.java

```
import org.omg.CORBA.*;

public class Serveur {
public static void main(String[] args) {
try {
    ORB orb = ORB.init( args, null );
    Icarrelmpl myobj = new Icarrelmpl();
    String ior = orb.object_to_string( myobj );
    System.out.println( ior ); orb.run();
}
catch(org.omg.CORBA.SystemException ex ) {
System.err.println( ex );
}
return ;
}
}
```

Exemple de code minimal CORBA en java

Client.java

```
import java.io.*;
import org.omg.CORBA.*;
public class Client {
public static void main( String args[] ) {
int nombre = 2;
int res;
try {
    ORB orb = ORB.init( args, null );
    String ior = args[0];
    org.omg.CORBA.Object obj = orb.string_to_object(args[0]);
    Icarre service = IcarreHelper.narrow(obj);
    res=service. carre (nombre);
    System.out.println("Res " + res);
}
catch(org.omg.CORBA.SystemException ex ) {
    System.err.println( ex );
}
}
```

Conclusion

- Solution non propriétaire
- Standard
- Libre choix de l'implémentation et du langage
- Solution fonctionnelle d'interopérabilité
- Plusieurs implémentations libres et open-source